

Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería
Ingeniería en Sistemas de información y Ciencias de la computación.
Curso - Algoritmos
Ing. Moises Eduardo Velasquez Oliva



Nombre del alumno: Vivian Reneé Rivera Kraemer

Carné: 0910-26-14699

Lugar y fecha: Jocotenango Sacatepéquez, 15/09/2026

Bitácora de Desarrollo – Proyecto "HojaClara"

=====

Semana: 1 (Fase 1: Análisis y diseño de la solución)

Fecha: 05 de Septiembre de 2026

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Esta semana me enfoqué completamente en entender el problema y planificar la solución antes de tocar una sola línea de código en JavaScript. Redacté el análisis preliminar sobre el caso de la Licenciada Silvia para tener claro que el objetivo es hacer una herramienta útil, sin internet y que calcule automáticamente. Además, trabajé en PSeInt diseñando los diagramas de flujo y el pseudocódigo de los cinco algoritmos centrales que pide la práctica: generación de cuadrícula, tokenización, evaluación matemática, recálculo en cadena y detección de referencias circulares.

2. Decisiones de diseño y por qué se tomaron

- **Diseño de la cuadrícula:** Decidí usar la estructura de ciclos anidados 'Para'. Un ciclo se encargará de las filas (números) y otro adentro se encargará de las columnas (letras). Elegí esto porque es la forma más directa de simular una matriz bidimensional.
- **Procesamiento de fórmulas sin usar `eval()`:** Como el documento prohíbe estrictamente usar la función `eval()`, tomé la decisión de tratar las fórmulas introducidas por el usuario como "cadenas de caracteres". Basándome en ejercicios anteriores del curso (como validar palíndromos), el diseño consiste en recorrer la fórmula letra por letra con un ciclo Para para separar los números de los operadores (tokenización).
- **Jerarquía de operaciones:** Para evaluar la expresión respetando la matemática básica, decidí estructurar el algoritmo evaluador de manera que primero busque y resuelva los signos de multiplicación y división usando condicionales Si/Entonces, y deje las sumas y restas para un segundo paso.

3. Problemas encontrados y cómo se resolvieron

- **Problema - El miedo al "recálculo infinito":** Al analizar cómo hacer el "recálculo en cadena", me topé con el problema de las referencias circulares (ejemplo: si el usuario pone en A1 que es igual a B1, y en B1 pone que es igual a A1). Me preocupaba que el programa se quedará en un ciclo infinito y trabajara la computadora.

- **Solución:** Resolví esto en el diseño de PSeInt creando un algoritmo preventivo de rastreo. Antes de calcular nada, el algoritmo guarda la "celda inicial" en una variable y empieza a saltar de dependencia en dependencia. Si en algún momento la celda que está revisando es igual a la celda inicial, el programa enciende una bandera (**hay Error <- Verdadero**) y detiene el proceso mostrando una alerta en pantalla, evitando que la aplicación colapse.

4. Siguientes pasos

Configurar mi repositorio de control de versiones (Git) como lo exige la práctica y empezar a traducir los ciclos anidados de PSeInt a JavaScript para generar dinámicamente la tabla HTML en el navegador (Nivel 1).

=====

Semana: 1 (Fase 2: Nivel 1 y Nivel 2 - Estructura de la cuadrícula y Estado)

Fecha: 06 de Septiembre de 2026

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Esta semana di el salto del diseño en PSeInt al código en JavaScript. Primero, definí formalmente la estructura de datos que almacenará el estado de la aplicación (Nivel 2). Luego, logré programar la generación dinámica de la tabla HTML (Nivel 1), traduciendo mis ciclos anidados **Para** de PSeInt a ciclos **for** en JavaScript. También inicialicé mi repositorio en Git y subí mi primer *commit* con la estructura base de los archivos (index.html, style.css, main.js).

2. Decisiones de diseño y por qué se tomaron

- **Estructura de datos basada en un Objeto (Diccionario):** Para guardar la información de la hoja, decidí no usar una matriz (array bidimensional). En su lugar, usé un objeto principal de JavaScript donde la clave es el nombre de la celda (como "A1" o "B2"). Tomé esta decisión porque hace que buscar y actualizar datos sea mucho más fácil y directo que calcular posiciones de filas y columnas [x][y].
- **Separación de propiedades por celda:** Decidí que cada celda en la memoria sea un objeto que guarda tres cosas distintas: el **valorOriginal** (lo que teclea el usuario), el **valorCalculado** (el resultado matemático) y una lista **dependeDe**. Tomé esta decisión preparándome para el Nivel 4 del proyecto, ya que me facilitará muchísimo el proceso de recálculo en cadena cuando la Licenciada Silvia modifique un dato.

3. Problemas encontrados y cómo se resolvieron

- **Problema - Conectar la pantalla con la memoria:** Me encontré con el problema de cómo hacer que el programa supiera a qué celda de mi estructura de datos correspondía cada "cuadrado" visual de la tabla HTML cuando el usuario le hace clic.
- **Solución:** Resolví esto inyectando un atributo `id` dinámico en cada etiqueta `<td>` de mi tabla HTML usando JavaScript. Así, cuando el ciclo `for` crea la primera celda de la primera fila, le asigna `id="A1"`. Luego, agregué un evento "escuchador" (`addEventListener`) que detecta el doble clic; cuando esto ocurre, el programa lee el `id` de la celda visual y sabe exactamente qué objeto de datos debe ir a modificar en la memoria para que el usuario empiece a editar.

4. Sigüientes pasos (Próxima semana)

Terminar de afinar la edición de celdas para que al presionar "Enter" el valor se guarde correctamente en la estructura de datos (cierre del Nivel 2) y empezar a programar el tokenizador en JavaScript para construir el evaluador de expresiones matemáticas (Nivel 3).

=====

Semana: 2 (Nivel 3: Motor Matemático)

Fecha: 09 de Septiembre de 2026

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

En esta fase se completó la implementación del motor matemático, permitiendo a la aplicación procesar operaciones aritméticas básicas (suma, resta, multiplicación y división). Paralelamente, fue necesario consultar material audiovisual y tutoriales para comprender la configuración y uso de Git, con el objetivo de integrar los commits correctamente y mantener un control de versiones estructurado.

2. Decisiones de diseño y por qué se tomaron

Debido a la restricción explícita de no utilizar la función nativa del sistema para evaluar expresiones, desarrollé un algoritmo de análisis léxico propio (`function analizarExpresion(exp)`). Se decidió estructurar la lógica de evaluación para que respete la jerarquía matemática estándar, procesando primero las operaciones entre paréntesis, seguidas de multiplicaciones y divisiones, y dejando las sumas y restas para la fase final del cálculo.

3. Problemas encontrados y cómo se resolvieron

Durante las pruebas iniciales, se detectó una inconsistencia crítica: el código implementado no se alineaba correctamente con la lógica planteada en los diagramas de flujo de la Fase 1, lo que generaba errores en cascada. Tras evaluar la situación, tomé la decisión de descartar el código defectuoso y reiniciar la programación de este módulo desde cero. Esta reestructuración total permitió que el flujo de ejecución coincidiera con el diseño original, garantizando la estabilidad del algoritmo.

4. Siguiendo pasos

Con el motor matemático funcional, el siguiente objetivo es establecer la interconexión entre las celdas para manejar referencias dinámicas (Nivel 4).

=====

Semana: 2 (Nivel 4: Dependencias y Referencias)

Fecha: 10 de Septiembre de 2026

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Logré que las celdas se comunicaran entre ellas. Ahora, si en una celda escribo "=A1+5", el programa va, busca el valor de A1 y me da el resultado correcto. Además, si cambio A1, la otra celda se actualiza solita en cascada.

2. Decisiones de diseño y por qué se tomaron

Para que la compu entienda que "A1" no es solo una palabra al azar sino la dirección de un cuadrado, usé Expresiones Regulares, que es como un buscador de patrones en el texto. Así el código detecta las letras seguidas de números y las cambia por el valor que tienen guardado en la memoria.

3. Problemas encontrados y cómo se resolvieron

Me costó un poco llevar el control de quién depende de quién. Lo solucioné creando un diccionario en la memoria llamado "dependencias" que anota qué celdas afectan a cuáles. Así, cuando modifico una, el programa sabe exactamente a quiénes más tiene que avisarles para que se recalculen.

4. Siguiendo pasos

Voy a empezar a trabajar en las funciones especiales como SUMA y en poder seleccionar varios cuadrillos a la vez para el Nivel 5.

Semana: 2 (Nivel 5: Funciones y Rangos)

Fecha: 10 de Septiembre de 2026

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Esta parte me emocionó mucho. Logré que la hoja entendiera palabras especiales como SUMA, PROMEDIO, MAX y MIN. Ahora puedo decirle que opere un rango completo, por ejemplo de A1 hasta C3, y junta todos los números de un solo golpe.

2. Decisiones de diseño y por qué se tomaron

Para poder recorrer un rango de cuadritos, decidí inventarme una función que convierte la letra de la columna en un número (A=0, B=1, etc.). Tomé esta decisión porque con números puros es facilísimo hacer un ciclo que vaya saltando de celda en celda recopilando la información.

3. Problemas encontrados y cómo se resolvieron

El problema principal era extraer exactamente la primera y la última celda del texto que metía el usuario. Lo resolví usando otra vez el buscador de patrones en el texto, dividiendo la fórmula en pedacitos para saber dónde empieza y dónde termina el rango antes de mandarlo a operar.

4. Siguiendo pasos

Tengo que ponerle un escudo a la página para que no se trabe si hago algo indebido, qué es lo que toca en el Nivel 6.

Semana: 2 (Nivel 6: Prevención de Errores)

Fecha: 11 de Septiembre de 2026 - 00:30 am

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Terminé de construir el sistema de alertas. Ahora la aplicación está protegida de cálculos imposibles o de ciclos infinitos que podrían congelar la computadora del usuario.

2. Decisiones de diseño y por qué se tomaron

Para manejar los problemas matemáticos normales, envolví mi código en una estructura de "intentar y atrapar" (try/catch). Si algo sale mal internamente, en lugar de que el programa colapse y se ponga en blanco, atrapa el error y muestra un mensaje amigable de "#ERROR!".

3. Problemas encontrados y cómo se resolvieron

Por fin enfrenté el famoso problema de las referencias circulares. Si ponía que A1 era igual a B1 y viceversa, la compu se quedaba pensando para siempre. Lo solucioné haciendo que el programa revise la lista de dependencias antes de calcular; si nota que una celda se está llamando a sí misma de forma infinita, se detiene y te pone "#CIRCULAR!" en pantalla. También le puse una regla rápida para que tire "#DIV/0!" si alguien intenta dividir entre cero.

4. Siguientes pasos

Ya casi termino. Toca hacer la parte de guardar los datos y poder descargar el archivo (Nivel 7).

=====

Semana: 2 (Nivel 7: Persistencia y Exportación)

Fecha: 11 de Septiembre de 2026 - 01:30 am

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Agregué los botones finales a la barra de herramientas. Ahora hay un botón para guardar toda la información y otro para exportar la hoja a un archivo CSV que se puede abrir en el Excel de verdad.

2. Decisiones de diseño y por qué se tomaron

Como la idea es que esto le sirva a la usuaria sin depender de internet ni de bases de datos complicadas, decidí usar la memoria local del navegador (**localStorage**). Así, la información se queda guardada en la misma computadora de la persona y si recarga la página por accidente, no pierde nada.

3. Problemas encontrados y cómo se resolvieron

Armar el archivo CSV fue un poco confuso porque tenía que juntar todo en texto plano separado por comas. Lo resolví usando ciclos anidados para que la compu fuera fila por fila y columna por columna pegando los valores y luego creando un archivo descargable directamente desde el navegador.

4. Siguientes pasos

El último reto: ponerle colores automáticos a las celdas y hacer que funcione con las flechas del teclado (Nivel 8).

Semana: 2 (Nivel 8: Navegación por Teclado y Estilos Visuales)

Fecha: 11 de septiembre de 2026 - 3:00 am

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Le di los toques visuales a la tabla para que cambie de color automáticamente si hay errores o números negativos. Además, ya se puede mover por toda la cuadrícula usando las flechas direccionales del teclado, lo que la hace sentir como un programa de Excel.

2. Decisiones de diseño y por qué se tomaron

Decidí dejar todo el tema de los colores en manos del archivo de estilos (CSS) creando clases específicas como `.cell-negative` y `.cell-error`. Así, el código de JavaScript solo se preocupa de decirle a la celda qué clase ponerse y el CSS hace que los negativos se vean rojos o que los errores tengan un fondo rojo claro.

3. Problemas encontrados y cómo se resolvieron

Lo que más dolor de cabeza me dio fue hacer que las flechas del teclado funcionan para moverse de izquierda a derecha. Como mis columnas son letras, no podía decirle a la compu que "le sumara 1 a la A". Tuve que reciclar mi función del Nivel 5 para pasar la letra a número, sumarle o restarle uno dependiendo de la flecha, y volver a convertirlo en letra para saber a qué cuadrito saltar.

4. Siguientes pasos

El proyecto ya está completo. Solo me queda navegar un poco por la aplicación para cazar algún bug suelto, limpiar mis comentarios de prueba y mandar el proyecto final.

Semana: 2 (Redimensionamiento Dinámico y Corrección de Errores)

Fecha: 12 de Septiembre de 2026

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Tras culminar los niveles principales del proyecto, se desarrolló una actualización para mejorar la usabilidad del sistema. Se implementó la capacidad de redimensionar la cuadrícula en tiempo real, permitiendo al usuario agregar o quitar filas y columnas de manera dinámica interactuando con los botones de la barra de herramientas.

2. Decisiones de diseño y por qué se tomaron

Para gestionar la modificación estructural, se decidió vincular los botones de la interfaz (como `add-row` y `add-column`) a detectores de eventos (`addEventListener`). Al hacer clic, el sistema modifica directamente el valor de las variables globales que controlan las dimensiones de la tabla y luego invoca la función principal de generación de cuadrícula. Se tomó esta decisión arquitectónica porque reutilizar la función generadora existente es más eficiente y limpio que intentar manipular o destruir nodos HTML individuales.

3. Problemas encontrados y cómo se resolvieron

Durante la fase de pruebas de esta nueva característica, se detectó una vulnerabilidad crítica: si el usuario presiona consecutivamente los botones de "Eliminar Fila" o "Eliminar Columna", las variables de dimensión llegaban a cero o a valores negativos. Esto provocaba que la cuadrícula desapareciera por completo de la pantalla, rompiendo la interfaz gráfica y generando errores de referencia en el código.

Para solucionar este fallo, introduje una estructura condicional de seguridad en los eventos de eliminación. Se implementaron las validaciones `if (ROWS > 1)` e `if (COLS > 1)`. Con esta corrección matemática, el programa audita el tamaño actual antes de restar una dimensión; si la tabla alcanza el tamaño mínimo de 1x1, el sistema ignora la instrucción de borrado, garantizando que el usuario siempre tenga al menos una celda activa en pantalla para trabajar.

4. Siguientes pasos

Integrar estas modificaciones al Manual del Usuario para instruir a la Licenciada sobre cómo expandir su área de trabajo de forma segura y dar por concluido el ciclo de desarrollo.

=====

Semana: 2 (Documentación y Manual de Usuario)

Fecha: 13 de Septiembre de 2026

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Una vez finalizado y estabilizado el código fuente de *HojaClara*, esta semana se inició y concluyó la redacción del Manual del Usuario. La documentación abarca desde los conceptos básicos de navegación por la interfaz gráfica, hasta la explicación detallada de la sintaxis para el ingreso de fórmulas matemáticas, el uso de rangos espaciales y el redimensionamiento dinámico de la cuadrícula.

2. Decisiones de diseño y por qué se tomaron

El enfoque principal para el diseño del manual fue la accesibilidad de la información. Teniendo en mente que la usuaria final (la Licenciada Silvia) requiere una herramienta práctica y directa, decidí estructurar el documento de manera progresiva: comenzando con las acciones más intuitivas (escribir datos y moverse con el teclado) hasta llegar a las características avanzadas (funciones de agregación y persistencia de datos). Se optó por un formato estructurado con viñetas y ejemplos claros en lugar de bloques densos de texto, para facilitar la lectura rápida y la consulta puntual.

3. Problemas encontrados y cómo se resolvieron

El mayor reto durante esta fase no fue técnico, sino de comunicación: ¿cómo explicar conceptos complejos de programación, como las referencias circulares o las validaciones estructurales de la cuadrícula, a un usuario sin conocimientos informáticos previos?

La solución fue aplicar un proceso de abstracción, eliminando toda la jerga técnica (como *ciclos infinitos*, *variables globales* o *nodos HTML*). En su lugar, traduje estos eventos basándome puramente en la experiencia visual y funcional del usuario. Por ejemplo, en lugar de explicar la lógica algorítmica detrás de la prevención de errores, el manual se enfoca en explicar qué significan las alertas visuales (**#CIRCULAR!**, **#DIV/0!**) y cómo el sistema protege el trabajo bloqueando acciones destructivas.

4. Siguiendo pasos

Con el software completamente funcional y la documentación de respaldo redactada y revisada, el proyecto *HojaClara* ha alcanzado su etapa de madurez. El paso final es empaquetar los archivos del código junto con el manual de usuario para proceder con la entrega formal de la solución.

Semana: 3 (Nivel 8: Selección Múltiple y Rango Visual)

Fecha: 15 de Septiembre de 2026

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Logré implementar la selección de celdas por arrastre usando el ratón. Ahora puedo hacer clic en una celda, mantener presionado y arrastrar sobre la tabla para marcar un bloque completo de cuadritos en color azul, tal como funciona en Excel o Google Sheets.

2. Decisiones de diseño y por qué se tomaron

Para que la computadora sepa qué celdas debe pintar, decidí convertir las letras de las columnas en números. De esta forma puedo calcular los límites del rectángulo usando el mínimo y máximo de

las filas y columnas seleccionadas (`Math.min` y `Math.max`), aplicando la clase CSS `.selected-range` a las celdas que quedan dentro de ese rango.

3. Problemas encontrados y cómo se resolvieron

Al principio, si soltaba el clic fuera de la tabla, el programa pensaba que todavía estaba arrastrando el ratón y seguía seleccionando celdas. Lo solucioné escuchando el evento `mouseup` directamente en todo el documento (`document`), garantizando que la selección se detenga de inmediato sin importar dónde suelte el botón del ratón.

4. Siguiendo pasos

Reparar errores en el pseudocódigo.

Semana: 3 (Nivel 8: Navegación por Teclado y Utilidades de Interfaz)

Fecha: 15 de Septiembre de 2026

Autor: Vivian Rivera / 0910-26-14699

1. ¿Qué se avanzó en esta etapa?

Agregué la navegación fluida con el teclado usando las flechas de dirección, Enter y Tab para moverse entre celdas rápidamente sin tocar el ratón. También creé la función del botón "Mayúsculas" en la barra de herramientas para convertir el texto de la celda activa a mayúsculas al instante.

2. Decisiones de diseño y por qué se tomaron

Para la navegación, usé una expresión regular en el evento `keydown` que separa el nombre de la celda activa en columna y fila (ej. "B" y "3"). Así puedo sumar o restar posiciones según la flecha presionada y cambiar el foco con `.focus()`. Para el botón de mayúsculas, decidí llamar directamente a la función `guardar Celda()`, logrando que el texto cambie y al mismo tiempo se evalúen las fórmulas y dependencias asociadas.

3. Problemas encontrados y cómo se resolvieron

Al presionar las flechas en los bordes de la tabla, el código intentaba buscar celdas que no existían (como la fila 0 o la columna -1) y tiraba error. Lo resolví agregando validaciones con `Math.max()` y `Math.min()` para fijar un tope y evitar que el cursor intente salir de los límites de la cuadrícula.

4. Siguiendo pasos

Realizar las pruebas de integración finales de todo el proyecto y empaquetar la documentación junto con los diagramas de flujo y pseudocódigo.